Evaluating Cross-Layer Integrity in Updates to Generated Clinical Interfaces

ANONYMOUS AUTHOR(S)

Data-backed interfaces can remain executable while updates change requested information. We examine reference-based integrity checks using 763 update pairs with eight injected fault types. Local checks accept every fault by construction; two cross-layer implementations reject every fault but share references and checking functions. A stress test shows that exact matching rejects 648 result-preserving SQL alternatives, while removing query equality lets 329 of 476 result-changing queries pass. Exploratory studies examine 20 independent updates, followed by six planned three-step sequences using the application pipeline. Time-window loss appears in both studies, but execution-based checks and simpler predicates make identical decisions. Three sequential attempts fail schema validation, leaving six steps unattempted. Multiple database states add no discrimination among observed outputs. These results expose limits of reference matching and generation contracts, without establishing an advantage for graph representation, general semantic validation, or clinical utility.

CCS Concepts: • **Human-centered computing** → **Human computer interaction (HCI)**; *Interactive systems and tools.*

Additional Key Words and Phrases: generative user interfaces, data-driven interfaces, electronic health records, integrity validation, incremental updates, traceability

ACM Reference Format:

Anonymous Author(s). 2027. Evaluating Cross-Layer Integrity in Updates to Generated Clinical Interfaces. In *Proceedings of 2027 CHI Conference on Human Factors in Computing Systems (CHI '27).* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Clinical records may contain more information than a person can review for a particular task. Selecting and arranging that information introduces choices about patient, item, period, source, and presentation. An interface can reduce what is shown while also hiding a mismatch between what was requested and what was retrieved. We investigate a technical way to expose such mismatches during updates. This paper does not measure a person's information-processing capacity or demonstrate a reduction in review burden.

For illustration, a preoperative assessment might summarize recent findings, whereas a follow-up view might combine selected trends and current values. These are motivating design examples, not validated clinical requirements or evaluation tasks. They motivate a data-backed interface whose information changes with a request, and the need to check the correspondence from request to presentation.

Existing technical checks leave a gap relative to that goal. SQL syntax, query execution, result schemas, interface code, and rendering can each be validated independently. Yet every artifact may pass while a query selects another patient, omits a time filter, leaves a result from the previous update, or connects a widget to the wrong data. Better text-to-SQL accuracy does not ensure that the same information need survives subsequent execution, connection, and presentation. A system can therefore work mechanically while returning information that does not answer the user's request.

We call a broken semantic correspondence between a scenario-grounded information need and the displayed information a **cross-layer inconsistency**. We evaluate a generated clinical interface as a process spanning an information need, SQL, an execution result, and a presentation, rather than as a collection of independently valid artifacts. Correctness

here does not mean that the complete clinical judgment is correct. It means that the patient, item, time, aggregation, source, presentation, connection, and version specified for an update remain consistent across layers.

Research on generative and malleable interfaces connects requests to interfaces through intermediate representations. Jelly maintains an evolving task-driven data model [2]; Bridging Gulfs exposes semantic design intent [9]; SimStep supports inspection and correction across task-level abstractions and evaluates fidelity across transformations [6]. Our comparison focuses on eight injected faults in request-to-data correspondence, applying four specified checking conditions to the same candidates.

We built a reference-based regression-testing framework for cross-layer inconsistencies in updates to clinical data-backed interfaces. EHRSQL-2024 supplies annotated questions and reference SQL [7]; MIMIC-IV Clinical Database Demo v2.2 supplies execution data [5]. The framework constructs minimal data and presentation artifacts and injects eight predefined fault types. The primary benchmark does not run a UI-generating model or test rendered screens; a separate constrained generation pilot is described in Section 5.8. A separately implemented oracle checks candidates against a contract: selected fields and expected values derived from the known target case. Both full validation conditions also receive that contract. Thus, the evaluation examines whether predefined checks cover injected faults when trusted reference information is available.

We ask:

RQ1 Which of the eight fault types can be constructed while preserving the local checks?

RQ2 How do local validation, artifact-level contracts, and cross-layer contracts differ in inconsistency leakage, valid-update acceptance, and fault-type coverage?

RQ3 For which rejected injections does a reported fault type enable one restoration from a saved valid artifact?

After freezing the evaluation method, we ran the test split once. We formed 763 update pairs from 134 question templates and performed 6,104 validations. Local validation accepted all inconsistent candidates because passing that condition is required for inclusion. Artifact-level contracts accepted 367; both full conditions rejected all inconsistent candidates and accepted all reference-valid candidates. The full conditions share core checking functions as well as expected values. Their agreement is an implementation consistency check, not independent evidence that storage representation has no effect.

An exploratory follow-up tests the reference dependence directly. Both full conditions reject all 648 query wrappers that preserve columns and typed result multisets on the same database. Reconstructed decisions without query equality accept these alternatives but also accept 329 of 476 result-changing equality inversions. Thus, acceptance of reference-derived candidates does not imply tolerance of valid alternatives.

The paper contributes a bounded evaluation artifact:

- A fault model for request, query, result, and presentation correspondence, with eight deterministic injection rules and explicit construction gates.
- A reproducible protocol that pairs reference-valid and inconsistent candidates, applies four specified checking conditions, and records detection, type reporting, and reference-assisted restoration.
- An analysis connecting implemented checks to detected and missed fault types, with explicit reference assumptions.

This is a technical evaluation. It does not evaluate the clinical validity of the scenario models, clinical safety, diagnostic accuracy, usability, cognitive load, clinical outcomes, or deployment. We treat technical correspondence and user benefit as separate evaluation targets.

2 RELATED WORK

2.1 Generative and Malleable User Interfaces

Generative-interface research has moved from producing screens from natural language or examples toward helping users understand, edit, and reuse generated artifacts. DirectGPT supports iterative work with large language models by combining conversation with direct manipulation and reference resolution on generated objects [8]. DynaVis combines conversational generation and editing of visualizations with direct interaction on the generated interface [11]. Both treat generated artifacts as objects that continue to change, rather than as one-shot outputs.

Intermediate representations provide structure for such updates. Jelly places a task-driven data model behind a generated interface and propagates task changes into data and views [2]. Bridging Gulfs uses semantic representations to narrow gaps among user intent, model interpretation, and interface output [9]. SimStep separates graphs for concepts, scenarios, learning objectives, and interfaces so that assumptions can be inspected and repaired across stages [6]. Across these systems, intermediate representations become points of contact for inspection and repair.

We study a narrower question: which specified checks cover faults injected into an update with known reference information? We do not compare initial generation quality or editing experience. The graph and tabular conditions provide two access paths to shared checking logic; they do not isolate the effect of storage architecture.

2.2 Inspection and Repair in Human-AI Collaboration

People do not simply accept or reject generated suggestions. Ignore, Trust, or Negotiate documents how clinicians ignore, trust, and negotiate AI suggestions, and argues for interactions that expose uncertainty and supporting information [10]. For generated interfaces, an undifferentiated error message may be less actionable than a diagnosis that identifies the broken relationship and the scope affected by a repair.

A rejection and an actionable diagnosis need not coincide. We record whether a validator rejects a fault, whether it reports the injected type, and whether a routine using that type and a saved valid artifact restores the candidate. The latter two measurements are coupled by the restoration procedure. Neither measures whether a person can locate or correct an error from the diagnosis.

2.3 From Natural-Language Questions to Data and Presentation

Text-to-SQL research evaluates whether a system can generate executable SQL that answers a natural-language question. EHRSQL-2024 provides a shared task for electronic health records that includes both answerable and unanswerable questions [7]. Its central challenges include generating SQL for answerable questions and abstaining when a question cannot be answered.

We do not compare SQL-generation performance. We use reference SQL as a starting point and examine inconsistencies that arise during subsequent updates. Even when a correct query was previously available, a question, query, result, or presentation can advance to a different version during an update. An executable query can also refer to a patient or time range that differs from the request. Validation must therefore cover not only question-to-SQL correspondence, but also SQL-to-result versioning, result-to-presentation compatibility, data-presentation connections, and update versions.

2.4 Model Transformation and Consistency

Bidirectional-transformation research has formalized how updates to structured sources and views should correspond. Lenses define laws for transformations between a source and its view [3]. Metamorphic testing has also been proposed

for model transformations where enumerating every correct output is difficult, using invariant relations instead of complete expected outputs [4].

The oracle problem concerns how a test distinguishes expected behavior from a fault; prior work includes specification-based, derived, and partial oracles [1]. Our implementation uses a restricted reference oracle: it requires equality of selected query, result, and binding fields from a known target. It does not implement lens laws or establish general semantic equivalence. The connection to consistency testing is methodological; the proposed artifact specializes fault injection and reference checking to boundaries in data-backed interface updates.

3 CLINICAL PROBLEM AND EVALUATION SCOPE

3.1 Scenario-Specific Information Needs

Our design process organized two clinical scenarios that motivated this work: a preoperative anesthesia clinic and follow-up after thyroid surgery. In the preoperative scenario, the surgery date anchors a one-time assessment that brings together patient demographics, the planned procedure, lifestyle and allergy information, medical and treatment histories, recent examinations, and prior anesthesia records. The clinician ultimately compiles a list of anesthesia risks. This scenario emphasizes a broad view of the patient's latest state.

In post-thyroidectomy follow-up, the visit date anchors the review. The clinician examines thyroid function, ultrasound findings, and levothyroxine replacement dose. In this design example, thyroid function and dose use a twelve-month trend, while other tasks use the latest values. Appropriate views therefore combine longitudinal trends with current summaries.

Figure 1 illustrates these differences and the scope of this paper. The scenarios are design examples, not clinically validated ground truth for our benchmark. Their task models have not completed expert validation. We use them to explain why the information and presentation retrieved from the same electronic health record must depend on a clinical scenario.

3.2 From Technical Correspondence to User Value

At least three conditions separate a generated clinical interface from value to a clinician. First, domain experts must define questions, required information, and completion criteria for a clinical scenario. Second, the system must translate the resulting information need into appropriate retrieval and presentation. Third, a clinician must use the presentation to review necessary information and perform the clinical task. This paper tests selected reference checks related to the second condition, using a known target; it does not evaluate that condition as a whole.

Our results cannot show that cross-layer integrity reduces verification effort or missed information. We examine selected technical checks separately from user experience. In a later user study, information correctness should be assessed alongside completion time and preference, so that a fast but incorrect response is not counted as a successful clinical task.

3.3 Defining an Information Error

Information can be wrong at several stages. This study does not determine whether source records are factually correct, whether a scenario's task model is clinically appropriate, or whether a clinician makes the correct judgment after viewing the interface. We define an **information error** within our scope as a state in which information delivered to the screen does not match a specified information need in at least one of eight dimensions: patient, item, time,

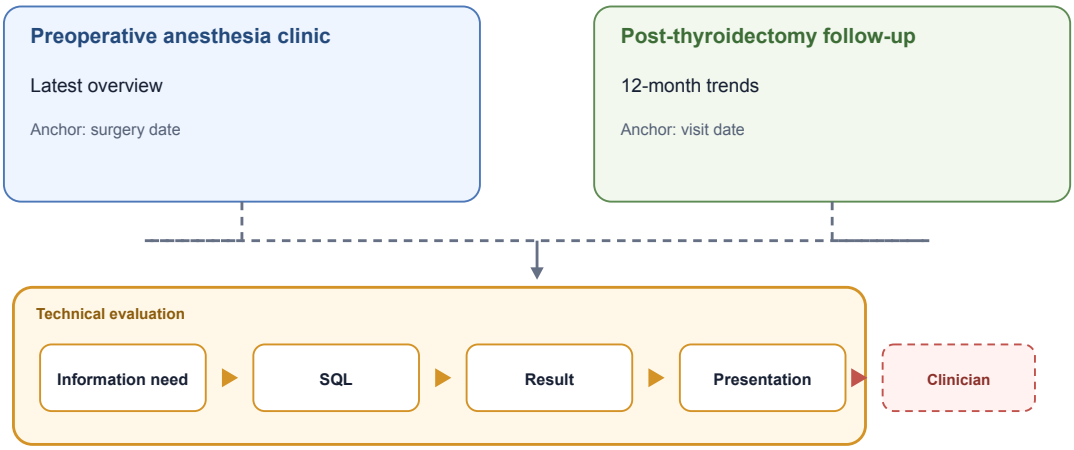


Fig. 1. Examples of scenario-dependent information and presentation. The preoperative clinic (left) summarizes patient history, surgical context, and recent tests relative to the surgery date. Post-thyroidectomy follow-up (right) combines 12-month trends in thyroid function and replacement dose with ultrasound findings relative to the visit date. This paper evaluates the technical path from information need to presentation; clinical validity and effects on clinicians require separate evaluation. The scenarios are not benchmark cases.

aggregation, source, presentation, connection, or version. A cross-layer inconsistency localizes such an error to a relationship between artifacts.

This definition distinguishes an ill-formed query from a well-formed query that retrieves different information. The local validator checks permitted SQL form and artifact compatibility. Detecting a mismatch with a request additionally requires a trusted expectation; the full conditions in this experiment receive that expectation from the target case.

4 DEFINING CROSS-LAYER INTEGRITY

4.1 Information Flow and Detection Mechanism

We model four layers: information need, SQL, execution result, and presentation component. An information need includes the target patient, clinical item, time, aggregation, source, and expected presentation. SQL translates that need into a database operation. An execution result has a version corresponding to the SQL, input, and time of execution. A presentation component refers to a result and displays it using a form compatible with its shape.

Figure 2 illustrates a single patient mismatch. The implementation derives expected values from annotated target cases and reference SQL, rather than extracting a validated clinical intent from arbitrary prose. It compares selected query, result, reference, and presentation fields and returns issue codes and type labels. The figure depicts the conceptual flow; exact checks and the privileged information used by the oracle and restoration routine are specified below.

For conceptual organization of the fault model, we express the required correspondences of an update candidate c as a conjunction. The implemented checks use selected reference fields; the equation is not a general semantic-equivalence test:

$$I(c) = P(c) \wedge D(c) \wedge T(c) \wedge A(c) \wedge S(c) \wedge V(c) \wedge L(c) \wedge R(c) \tag{1}$$

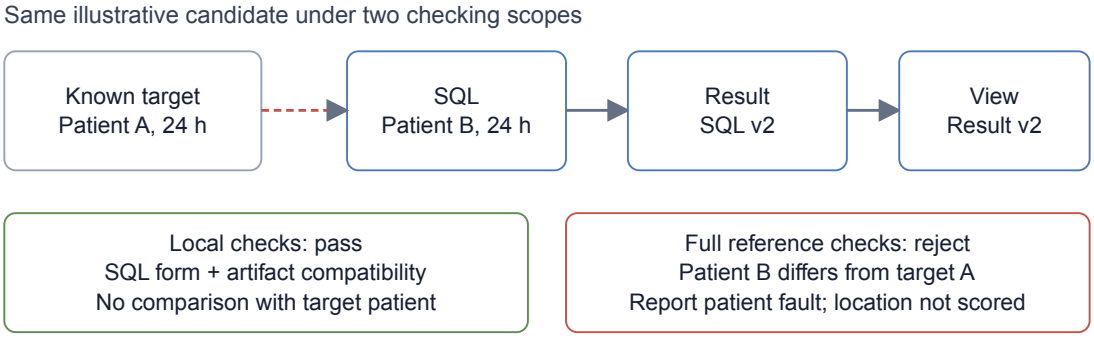


Fig. 2. Illustrative patient mismatch, using invented labels. The local validator checks permitted SQL form and artifact compatibility without comparing the requested patient. The full checks compare the candidate against a known target contract. Patient B differs from reference Patient A; the 24-hour interval is unchanged. The diagnosis can report a patient fault, but the evaluation does not score the exact node or edge location.

Table 1. Eight cross-layer inconsistencies generated for evaluation

Type	Mutated element	Properties preserved during construction
Patient	Patient condition in SQL	Syntax, read-only, execution
Clinical item	Selected column or target	Syntax, execution, result type
Time	Point or interval condition	Syntax, execution
Aggregation	Count, maximum, minimum, mean, etc.	Syntax, execution, result type
Source	Recorded provenance metadata	SQL remains unchanged
Presentation	Component used to show result	Reference and shape checks
Connection	Data node referenced by component	Per-node type
Stale result	Result version after SQL update	Existing result schema

P denotes patient, D clinical item, T time, A aggregation, S source, V presentation form, L the data-presentation link, and R the correspondence between SQL and result versions. These symbols organize the fault model rather than specify the exact implementation of the oracle. Each inconsistent candidate receives one injection rule; that mutation may trigger several failed checks. Passing the selected reference checks is narrower than satisfying general semantic equivalence.

4.2 Eight Inconsistency Types

Table 1 defines the eight inconsistencies. A patient inconsistency preserves the patient in the information need while changing the patient condition in SQL to another valid candidate. An item inconsistency changes the selected column or aggregation target. A time inconsistency changes a condition associated with a time point or interval. An aggregation inconsistency replaces a count, maximum, minimum, average, or related operation with another valid operation.

A source inconsistency changes the recorded provenance-table metadata while leaving SQL and its result fixed. It does not execute a query against an alternative table. A presentation inconsistency changes the baseline component type while preserving SQL and data: a metric becomes a dataframe, or a dataframe becomes another table component. The label denotes violation of a predetermined presentation rule; it does not establish that the alternative is misleading

or clinically inappropriate. A connection inconsistency redirects the widget to the source data node with compatible columns and mapping. A stale-result inconsistency attaches the source result to the target query when the two result digests differ.

These categories are not an empirical distribution of errors observed in clinical practice. They form a fault model that makes specific relationships testable. Because we inject one inconsistency into each candidate, the results do not characterize performance on compound failures.

5 EVALUATION FRAMEWORK

5.1 Constructing Baseline Cases

The inputs are answerable questions and reference SQL from EHRSQL-2024. Unanswerable cases lack reference SQL from which to construct a baseline interface, so we exclude them from the primary evaluation while recording their count. Reference queries are executed read-only against the SQLite version of MIMIC-IV Demo v2.2. We reject statements that write data, contain multiple statements, or use disallowed syntax before execution, and impose a five-second limit on each query.

We use a metric component when an execution result is a single value and a dataframe when it contains multiple rows or columns. Empty results are retained as results with a defined column structure. Each baseline case stores annotated patient, item, time, and aggregation fields associated with the question template, parsed SQL, result shape, data node, presentation component, and version identifiers for these artifacts.

We group cases by question-template string and consider ordered source-target pairs within each group. For each template and fault type, a seeded hash orders eligible descriptors; we retain the first pair that passes construction checks. Thus, there is at most one pair per template and type. A template can contribute to several types, and the same underlying source or target case can recur. These are constructed transitions, not observed sequences of user edits.

For inclusion, a valid candidate must pass the reference oracle, its injected counterpart must fail that oracle, and the inconsistent candidate must pass local validation. This last gate intentionally concentrates the benchmark on locally admissible faults and makes 100% local leakage true by construction. It is not an estimate of failure prevalence or of local-check accuracy on naturally generated candidates.

5.2 Reference Oracle and Its Dependence on the Contract

The oracle is implemented separately from mutation generation and validator dispatch, but it uses the same target-case contract. It checks the question checksum, normalized reference-SQL checksum, provenance-table list, result-query checksum, result digest, column checksums, widget identifier, data-node binding, fields, and component type. This is equality of selected reference fields, not an independent judgment of clinical intent or SQL semantic equivalence.

The oracle does not compare the complete JSON document. It nevertheless requires the selected identifiers, column order, SQL checksum, and other contract fields to match. A semantically equivalent alternative query or presentation may fail these reference checks. Valid candidates in this experiment are constructed from the reference itself; their acceptance cannot estimate acceptance of diverse valid solutions. Separate code paths reduce some implementation coupling, but common reference errors can affect generation, the oracle, and full validation together.

Table 2. Four validation conditions

Condition	Individual artifacts	Cross-layer relations	Access
Local validation	Basic checks	Reference existence; shape	Artifacts
Artifact-level	Detailed checks	Partial	Artifacts
Graph-based	Same lower checks	Target reference	Linked access
Tabular	Same lower checks	Same target reference	Flat access

5.3 Four Validation Conditions

We apply the same candidate to the four conditions in Table 2. Local validation checks SQL syntax and permitted read-only form, node-reference existence, a nonempty column schema, widget-field membership, and metric-shape compatibility. Query execution occurs during candidate construction, not during the timed validator call. No browser rendering is checked. Artifact-level contracts additionally compare recorded provenance with tables parsed from SQL, the result's query checksum with current SQL, and the widget type with the type assigned from result shape. Despite its short name, this condition includes these adjacent-artifact checks. It does not compare the target request annotations or target binding against the full reference contract.

The graph-based cross-layer contract follows the question, target data artifact, and widget references and compares them with the target contract. The tabular cross-layer contract first compares flat fields against the same expected values. Both then call the same semantic-query, target-data, and target-widget checking functions. The semantic-query function checks annotated literals or normalized fragments and also requires equality with the normalized reference-SQL checksum. Neither implementation infers correctness from unrestricted natural language. These are two access paths over shared checking logic, not independently implemented storage architectures. Their agreement cannot establish a causal effect of storage form.

5.4 Fault-Type Reporting and Reference-Assisted Restoration

A diagnosis contains an issue code, a layer label, and sometimes a fault-type label. We score fault-type coverage when the injected type is present in the set of reported type labels. We do not require a unique diagnosis, penalize extra type labels in this score, or verify an exact node or edge location. We therefore interpret this metric as fault-type coverage.

The restoration routine receives the injection type and the saved valid candidate. If the reported labels include the injected type, it copies the complete target data artifact for SQL, provenance, or stale-result faults, or the complete valid widget for presentation and connection faults. Otherwise it returns the candidate unchanged. The restored candidate must pass both the same validator and the reference oracle.

This procedure tests recovery with privileged reference information. It does not infer the injected type solely from the diagnosis, synthesize a minimal patch, or discover a correct state from an unknown error. Restoration attempts are gated by the known invalid label and validator rejection. Consequently, this protocol does not measure the risk of modifying an incorrectly rejected valid update.

5.5 Evaluation Protocol

Figure 3 summarizes the protocol. On development data, we prioritized 50 cases with distinct question templates to verify SQL execution and baseline conversion. On validation data, we checked candidate counts per inconsistency type, the number of represented templates, label agreement, and the requirement that saved outputs contain no sensitive

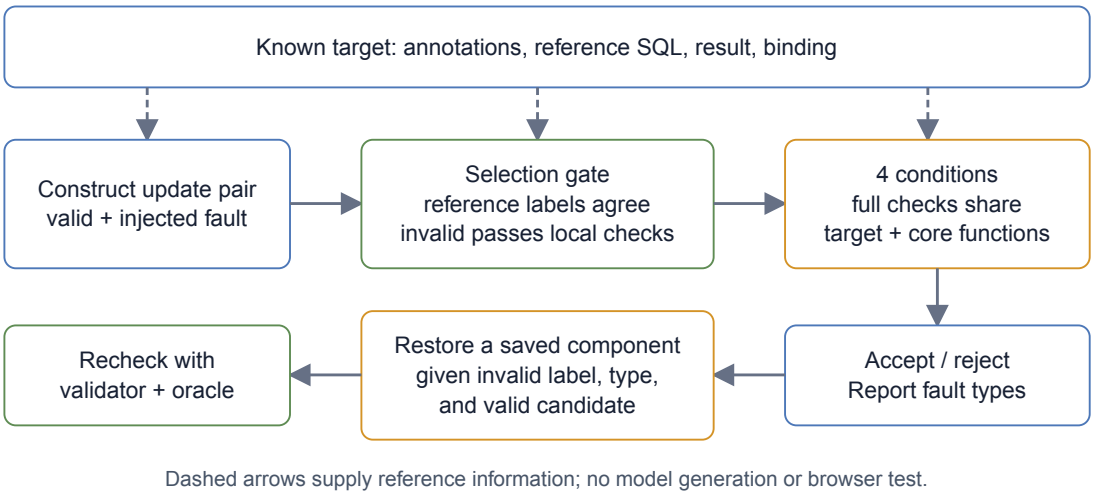


Fig. 3. Evaluation protocol and reference information. Candidates are retained only if their labels match the reference oracle and the inconsistent candidate passes local checks. The oracle and both full conditions use the same target contract; the full conditions also share core checks. Restoration receives the known invalid label, injected type, and saved valid candidate, and must pass the same validator and oracle afterward.

data. A planned column-order mutation could not be generated, so before inspecting the test split we replaced it with the presentation mutation. We left all other mutations, metrics, analyses, and stopping rules unchanged.

After the validation-data criteria were met, we froze the evaluation code, configuration, and analysis in a versioned execution manifest. We then ran the test split once. No generative or external model was called during evaluation.

5.6 Metrics and Statistical Analysis

The primary metrics are **inconsistency leakage rate** and **valid-update acceptance rate**. Leakage is the fraction of injected inconsistent candidates accepted; valid acceptance is the fraction of reference-valid candidates accepted. Secondary metrics are fault-type coverage, reference-assisted restoration success, and mean validation time per candidate. Type coverage divides correctly reported injections by all 763 injections. Restoration is reported both per attempted restoration and per injected fault.

We calculate paired condition differences and use 10,000 question-template cluster bootstrap resamples for 95% intervals. This retains dependence among candidates from one template. It does not address shared patients, overlapping cases, or common injection rules across templates. The intervals describe variation in the retained template sample under the fixed fault model, not uncertainty about natural error rates or new hospitals. Degenerate intervals arise when all resampled clusters have the same contrast; they are not proof of universal performance.

5.7 Exploratory Checks of Reference Dependence

After inspecting the primary results, we designed two exploratory analyses and froze their rules before execution. First, we reconstructed acceptance from the saved diagnostic codes in the test and validation logs. The validator collects all

applicable issues and accepts exactly when none remain. Removing selected codes therefore reconstructs acceptance decisions, but does not rerun an ablated validator or estimate diagnosis, restoration, or latency effects.

Second, we applied the unchanged validators to the 934 answerable test cases using four query forms: the original query, a whitespace control, a wrapper of the form `SELECT * FROM (original) WHERE 1 = 1`, and one equality-to-inequality change between a column and literal in a `WHERE` clause. The latter changes the first applicable equality in depth-first syntax-tree order, preserving all literals and other query text. We retain the reference widget and bind each executed result to its actual query. We do not require candidates to pass local validation for inclusion.

A separate comparison executes queries on read-only SQLite and compares column names and multisets of type-tagged cell values without using the reference oracle or its result-digest function. A wrapper is counted as a valid alternative only when these observations agree. An equality inversion is counted as result-changing only when columns agree and the result multiset differs; unchanged results are reported separately. The reference SQL remains a proxy for the requested information, not a clinical judgment. This comparison concerns one database state and ignores row order.

We exclude non-read-only or unparseable queries, nondeterministic functions, and limits without an ordering clause. Execution has a five-second query limit and a 10,000-row cap; failures and exclusions retain their own counts. Original-query repetition checks result stability. We report acceptance, issue codes, and denominators, including reconstructions that remove query equality, annotation checks, or both. These finite, exploratory case sets do not support estimates of natural generation-error frequency.

5.8 Exploratory Pilot with Model-Generated Updates

After the stress test, we froze a separate, constrained generation pilot. Twenty purposively authored requests independently modify one synthetic interface containing weight and pulse displays. They change time windows, patients, latest-item counts, units, measures, ordering, or visualization. The prompt explicitly preserves unspecified conditions and an unchanged companion widget. Each request starts from the same initial interface; these are not sequential updates or clinician-authored tasks.

Gemini 2.5 Pro generated one SQL-and-widget JSON artifact per request through Vertex AI, with temperature 0.2, top-p 1, a 4,096-token output cap, a 1,024-token thinking budget, and seeds 20260908–20260927. We made no repairs or retries. This compact artifact uses the existing Streamlit widget renderer but bypasses the application's scope-selection and incremental graph-generation pipeline. Only synthetic schema, initial queries, and requests were sent to the model; target SQL and database rows were withheld.

We compare structural and execution checks, strict target-SQL matching with display checks, and execution-based checks on one or three synthetic database states. The latter interpret manually specified change and preservation conditions in Python and compare ordered rows, units, bindings, and display settings; they do not access reference SQL. For audit labels, a separate SQL implementation evaluates the same conditions on four other states. All states contain temporal boundaries and distractors; selected states also have missing values or no measurements in the requested interval. The audit shares requirement specifications and display-checking code and is not independent expert ground truth. Finite-state agreement is not SQL equivalence. Execution-based SQL evaluation has precedents [12]; we do not claim a new general SQL evaluation method.

We retain all returned outputs, including malformed responses, and record API failures separately. These conditions differ from the primary benchmark's validators. They do not isolate graph representation or establish population error rates. A simpler time-filter diagnostic was designed only after inspecting the outputs and is reported separately.

5.9 Follow-Up through the Incremental Application Pipeline

We then froze six purposively authored three-step sequences that combine window, patient, latest-count, unit, aggregation, and chart changes. They use the application's existing scope selection, prompt construction, graph normalization, and result attachment. A process-local synthetic DWH model, restricted SQLite connection, and recording SDK client replace external dependencies; application update code remains unchanged. Gemini 2.5 Pro uses temperature 0.2, top-p 1, an 8,192-token output cap, a 1,024-token thinking budget, and one attempt per call. Requests expose schema and the current edit, while cumulative target specifications and database rows remain withheld. Every sequence starts from the same initial graph.

Completed states feed the next update without semantic repair. Audit targets accumulate the authored requests, so later failures may propagate earlier drift rather than represent new errors. A pipeline failure stops that sequence and leaves dependent steps unattempted. We preserve all attempted terminal states and separately count completion and requirement conformance. Structural checks include actual cached-result consistency. A simple syntax-tree baseline checks mandatory patient, item, and time predicates, aggregate kind, non-null counts, unit conversion, ordering, and limits. It supports inclusive BETWEEN and constant datetime expressions, but is not a complete SQL semantic analyzer. All informed conditions share display and binding requirements. Execution checks use one or three states; audit SQL uses four other states. As before, shared authored requirements and fixture construction limit independence. We report paired decisions and the first failure per sequence, without treating steps as independent samples or estimating population rates.

5.10 Data Management and Reproducibility

EHRSQL-derived released artifacts do not contain raw data, question text, reference SQL, patient identifiers, or patient-level result values. They contain hashed case identifiers, input and candidate checksums, inconsistency type, decision, diagnosis, restoration outcome, aggregates, and execution environment. We froze formal results as JSON and JSON Lines, then converted them to TSV in an independent procedure that recomputed counts, primary metrics, and confidence intervals. The recomputed values matched the original summary. The separate generation pilot releases its fully synthetic fixtures, requests, model responses, token usage, and checksums.

6 RESULTS

6.1 Baseline Cases and Candidates

The test split contained 1,167 cases. We evaluated the 934 answerable cases with reference SQL and excluded 233 unanswerable cases from the primary evaluation. Reference SQL executed and baseline artifacts were constructed successfully for all 934 cases; 82 yielded empty results with defined column structures. We formed 763 before-and-after update pairs from 134 question templates. For each pair, we created one valid and one inconsistent candidate and applied four conditions, yielding $763 \times 2 \times 4 = 6{,}104$ validations. Candidate labels disagreed with the reference oracle in zero cases.

We generated 52 patient, 55 item, 106 time, 34 aggregation, 134 source, 134 presentation, 129 connection, and 119 stale-result inconsistencies, totaling 763. For patient, item, time, and aggregation mutations, we skipped cases where no alternative SQL could change exactly one semantic dimension. Candidate counts therefore differ across types and depend on the data and question forms.

Table 3. Inconsistencies accepted by artifact-level contracts

Type	Generated	Leaked	Leakage rate
Patient	52	51	98.08%
Clinical item	55	51	92.73%
Time	106	103	97.17%
Aggregation	34	33	97.06%
Source	134	0	0.00%
Presentation	134	0	0.00%
Connection	129	129	100.00%
Stale result	119	0	0.00%
Total	763	367	48.10%

Table 4. Primary and secondary results across four conditions

Condition	Leakage	Valid accepted	Type covered	Mean time
Local validation	763/763	763/763	0/763	2.152 ms
Artifact-level	367/763	763/763	387/763	7.441 ms
Graph-based	0/763	763/763	763/763	9.339 ms
Tabular	0/763	763/763	763/763	9.296 ms

Construction recorded 3,420 failed pair-building attempts: 1,140 for patient, 1,383 for item, 576 for time, and 321 for aggregation; none were recorded for the other types. These count attempted constructions, not distinct excluded questions or patients. Retaining only the first successful pair per template and type selects faults that this data and construction procedure can express.

6.2 RQ1: Inconsistencies That Pass Local Validation

Local validation accepted all included inconsistent candidates, as required by the construction gate. This confirms that the gate operated as specified; it is not an empirical discovery that local validation misses all real errors. The comparison asks which additional checks detect these deliberately selected faults.

Leakage under artifact-level contracts varied by inconsistency type (Table 3). Its three additional checks rejected every provenance-metadata, presentation-rule, and stale-result fault. In contrast, 92.73% to 98.08% of patient, item, time, and aggregation faults leaked, as did every connection fault. These checks omit target-request and target-binding comparisons; some semantic injections were nevertheless rejected because the resulting artifact also violated a checked shape or mapping rule.

6.3 RQ2: Detection, Acceptance, and Type Coverage

Table 4 presents the primary and secondary results. The graph-based and tabular cross-layer contracts rejected all 763 inconsistent candidates and accepted all 763 valid candidates. Compared with local validation, the leakage difference was −100.00 percentage points with a 95% confidence interval of [−100.00, −100.00]. Compared with artifact-level contracts, it was −48.10 points, with a 95% confidence interval of [−49.56, −46.53]. Valid-update acceptance was 100.00% for all four conditions, yielding no condition difference.

Artifact-level contracts rejected 396 inconsistent candidates and reported the injected type in 387, giving 50.72% coverage over all 763 injections. Both full conditions included the injected type in their reports for all cases. This score does not establish exact location accuracy or diagnosis precision. Mean validation times were 2.152, 7.441, 9.339, and 9.296 ms for local, artifact-level, graph-based, and tabular conditions. These times exclude query execution, mutation generation, restoration, and rendering. The 0.043 ms difference between the full conditions is descriptive; no repeated-run performance comparison was conducted.

6.4 RQ3: One-Step Reference-Assisted Restoration

Local validation rejected no inconsistent candidate and attempted no restoration. Artifact-level contracts triggered 396 attempts; 387 passed both the validator and reference oracle, or 97.73% of attempts and 50.72% of all injections. In the other nine cases, the reported types did not include the injected type, so the restoration routine returned the unchanged candidate.

Both full conditions enabled restoration of all 763 injections using the saved valid data artifact or widget. Because restoration is gated by correct type reporting and copies a known valid component, these successes are coupled to the type-coverage result. They do not constitute an independent demonstration of automatic repair capability. No valid candidate was rejected, and the protocol would not attempt restoration of a candidate labeled valid even if it were rejected.

6.5 Exploratory Results on Reference Dependence

Replaying the saved diagnostic codes produces zero leakage after removing either query equality or annotation checks alone: 0/763 on test and 0/758 on validation. Removing both produces 238/763 and 237/758 leakage, respectively. Artifact-level checks plus query equality and target binding reproduce the full graph's acceptance decisions on all 3,042 valid and inconsistent candidates across these splits. These are acceptance-only reconstructions, not measurements of diagnostic or representation equivalence.

The SQL stress test excludes 286 of 934 cases containing current-time expressions under the fixed nondeterminism rule. The remaining 648 cases cover 126 templates. Original queries and whitespace controls are accepted under every condition. All 648 wrappers preserve column names and typed result multisets, yet both full conditions reject every wrapper solely with `target_query_mismatch`. These are false rejections under the result-based criterion in Section 5.7, although they correctly enforce the stricter reference-encoding contract.

For equality inversion, 60 cases have no applicable equality, 42 change column names, and 70 retain the same result; we do not score these as result-changing faults. The remaining 476 cases span 96 templates and change the result while preserving column names; 147 yield empty results. No execution failure or row cap occurs. Table 5 reports the retained candidates. Reconstructed decisions without query equality let 329/476 result-changing candidates pass. Annotation checks report no mismatch on these candidates; the 147 remaining rejections arise from result-shape checks, including 143 also caught locally.

6.6 Generated Outputs Expose a Preservation Failure but No Added Discrimination

All 20 model calls returned parseable artifacts in approximately 194 seconds. Eighteen conformed to the finite audit; two requests for the latest one or three weight measurements dropped both time bounds. The prompt required preserving the existing one-day interval and fixed upper timestamp. Both outputs displayed a synthetic row later than that timestamp. Structural checks accepted both outputs, while the one-state and three-state execution checks rejected them and

Table 5. Exploratory SQL stress test. The last three rows reconstruct acceptance by removing graph diagnostic codes; they do not rerun modified validators. Denominators exclude the separately reported cases.

Condition	Valid wrappers rejected	Result changes accepted
Local validation	0/648	333/476
Artifact-level contracts	0/648	329/476
Graph contract	648/648	0/476
Tabular contract	648/648	0/476
Graph codes minus query equality	0/648	329/476
Graph codes minus annotations	648/648	0/476
Graph codes minus both	0/648	329/476

accepted all 18 conforming outputs. Strict reference matching rejected three conforming outputs: two used an integer rather than decimal multiplication constant with a REAL column, and one omitted redundant aliases.

After observing the failures, we added a diagnostic that checks mandatory inclusive time predicates in the SQL syntax tree, together with structural validity. It made the same decisions on all 20 outputs as both execution-based checks. It does not validate patients, units, or arbitrary SQL forms. Because both observed errors concern the same temporal-preservation failure, the pilot provides no evidence of added discrimination over this simple check or from additional database states. This post-hoc result limits the interpretation; it is not a preregistered comparison.

All 20 artifacts rendered without application exceptions or warnings. We compared the data passed to 39 tables and one line chart with their SQL results, including table columns and chart axes. Rendering success therefore coexisted with the two requirement violations. This verifies rendering data, not visual usability or clinical correctness. The pilot supplies a concrete preservation failure and reference-matching limitations, not evidence of general robustness.

6.7 Sequential Updates Propagate Drift and Expose Generation Failures

The follow-up made 24 model calls in approximately 207 seconds without API errors. Of 18 planned steps, 12 were attempted: nine completed and three failed application schema validation, leaving six dependent steps unattempted. Seven completed states conformed to the finite audit. In one sequence, requesting the latest three records removed the previously requested seven-day window; the subsequent unit conversion preserved that incorrect query scope. These are two nonconforming states in one sequence, not two independent origins of drift. Structural checks accepted both. The simple SQL, one-state, and three-state checks made identical decisions across all 12 attempts: each accepted the seven conforming states and rejected the two completed violations and three pipeline failures. Neither broader execution nor additional states improved discrimination in this sample.

The failed average, count, and chart edits all returned a Markdown widget without content or a data binding. A post-hoc diagnosis found that these responses satisfied the JSON schema actually sent to the model but violated an application-level Pydantic validator. The chart request also conflicted with a generation plan that retained the existing dataframe type; this is a documented prompt inconsistency, not an experimentally established cause of that response. In all three failures, the final graph and context remained unchanged. Replaying recorded responses reproduced all 12 attempts and intermediate states without new model calls. All 12 terminal screens rendered, and the data passed to their 24 display tables matched the recorded cache. This includes retained old screens after failed edits and does not imply successful task completion. The result motivates fixing generation-contract inconsistencies before expanding this evaluation; it does not establish superiority of the proposed checks.

7 DISCUSSION

7.1 What the Controlled Comparison Establishes

The benchmark identifies gaps between the specified checks under a known-target assumption. Artifact-level checks detect provenance metadata, baseline presentation mapping, and query-result version mismatches. They can also reject a semantic-query mutation when it changes result shape, without reporting the injected semantic type. The full conditions add target-query and binding checks and reject all retained injections.

This is evidence about coverage of the implemented fault model. Both the fault generator and the check designer know the relationships to change or preserve. The diagnostic replay shows that query equality and annotation checks overlap on that suite. The exploratory stress test separates them: exact-query matching rejects result-equivalent wrappers, while retaining annotation checks without exact-query matching misses 329 result-changing candidates. Removing query equality alone is therefore not a sufficient remedy. The current target-result comparison is also conditional on matching the reference query version. A broader contract must specify which alternative queries and outputs are permissible, then be evaluated on cases not used to design it.

7.2 Implications for Data-Backed Interface Updates

The practical design question is where a system obtains trustworthy expectations. A stored binding or query-result version may be available after an update, while the correct SQL for a new natural-language request may not be. Our full validators receive both. A system that generates expectations with the same model that generates its output can produce mutually consistent but jointly wrong artifacts. The present evaluation does not resolve that problem.

For developers, the fault model provides a way to specify which reference fields a check needs and which change it can detect. It also makes a distinction visible: provenance metadata disagreeing with SQL is different from SQL querying an inappropriate source, and violating a baseline widget rule is different from misleading a user. Evaluations should name those distinctions rather than combine all rejections as evidence of clinical correctness.

7.3 What the Graph and Tabular Agreement Means

Both full conditions access the same target contract and call shared query, data, and widget checks. Their identical results show that the two tested access paths give consistent outcomes on this suite. They do not establish storage-independent performance, scalable graph propagation, or an advantage for either representation. Those questions require independent implementations and measurements of their intended tasks.

Prior generative-interface work uses intermediate representations to support inspection and editing [2, 6, 9]. Our benchmark focuses on a narrower developer-facing problem: checking data correspondence against a reference. Whether those checks improve how people inspect and edit generated interfaces remains open.

7.4 Evaluating Use in Generated Interfaces

The present artifact is designed for developer regression checks. Extending it to generation-time validation requires testing naturally generated failures and the availability of trusted expectations. The illustrative clinical scenarios do not establish how often the eight faults occur or how consequential they are in practice.

A subsequent technical study should freeze validators before collecting naturally generated updates, include valid alternatives, obtain reference judgments independent of validator design, and compare against established checking methods. If the contribution concerns clinician verification, an additional study should compare the same interface

with and without diagnoses, using expert-reviewed scenarios and reporting faulty-update acceptance, valid-update acceptance, review time, and repair quality separately. Such studies may show a benefit, a cost, or no effect.

7.5 Restoration Is a Different Capability from Repair

Copying a saved valid artifact is useful when a trusted target state exists, but it does not solve correction of an unknown generated state. Our routine also receives the injected type, and replaces a whole data artifact or widget rather than computing a minimal patch. A practical repair evaluation must hide injection metadata, withhold the complete valid target, test collateral changes, and judge correctness independently. The current 100% restoration result should not be used as a claim that a diagnosis alone supports reliable automatic repair.

8 LIMITATIONS AND ETHICS

First, we inject one inconsistency at a time using deterministic rules. The generated candidates do not represent the distribution of errors that a generative model produces naturally. A condition that achieved 100% on single faults may not perform similarly on compound failures, ambiguous requests, or unknown presentation components. Future work should apply the frozen validators to naturally occurring candidates and qualitatively analyze failures outside the taxonomy.

Second, the benchmark uses the known target query and its executed result, not merely a previously valid query. It therefore assumes that a trusted state is available for the requested update. This assumption is restrictive for generative interfaces. Question interpretation, initial generation, abstention, and alternative valid realizations require separate evaluation.

Third, the primary benchmark uses MIMIC-IV Demo v2.2 and EHRSQL-2024 question templates. The preoperative anesthesia and post-thyroidectomy scenarios illustrate the problem but are not formal evaluation cases. Candidate counts differ across inconsistency types because result shapes and valid alternatives differ. We cannot generalize the findings to other clinical scenarios, data models, institutions, complex visualizations, or updates spanning multiple sources. Applicability to non-healthcare data and workflows is also untested.

Fourth, full validation and the oracle share the target reference; graph and tabular implementations additionally share core checking functions. This common dependency limits evidence from agreement. The exploratory reconstructions concern acceptance only. The SQL stress test uses one wrapper rule and one equality inversion on a single database state, excludes 286 time-dependent cases, and ignores row order. Its comparison code avoids the original oracle and digest, but still shares the reference SQL and SQLite engine. The observed false-rejection and leakage counts cannot estimate performance across naturally generated queries or all semantically equivalent interfaces.

Fifth, this is a technical evaluation on publicly available deidentified demo data and fully synthetic pilot fixtures. Reference SQL is a technical baseline, not a clinically validated definition of appropriate presentation. We did not measure effects on verification burden, acceptance of faulty updates, or clinical judgment. We make no claims about clinical safety, diagnostic accuracy, usability, cognitive load, clinical outcomes, deployment, or generalization across institutions. Any human-participant study will be separate work conducted only after applicable ethics review, permission, recruitment, consent, withdrawal, and data-management procedures are established.

Released artifacts contain no question text, reference SQL, patient identifiers, or patient-level result values. We state the conditions of use and provenance for MIMIC-IV Demo and EHRSQL-2024 and include only releaseable checksums, categories, decisions, and aggregate statistics in reproducibility materials.

Both generation studies use one model and purposively authored requirements. The initial pilot bypasses the full application pipeline; its two failures belong to one request family, and a post-hoc time-filter check explains all observed discrimination. The follow-up uses the existing updater with synthetic dependencies, but only nine of 18 planned steps complete because three generation failures prevent six dependent attempts. Its two completed violations belong to one sequence, and the pre-fixed simple baseline matches both execution checks. Shared specifications, fixture construction, and display checking limit audit independence. These results do not establish improved detection over strong existing validators, tolerance of arbitrary equivalent programs, or benefits to clinicians.

9 CONCLUSION

We present a reference-based regression-testing artifact for request, query, result, and presentation correspondence in clinical interface updates. Eight deterministic injection rules yielded 763 paired updates and 6,104 validations on EHRSQL-2024 and MIMIC-IV Demo. The artifact makes construction gates, expected fields, diagnostic scoring, and restoration inputs explicit.

All injected candidates passed local checks by construction. Artifact-level contracts accepted 48.10% of them. The two full conditions rejected every injected inconsistency and accepted every reference-valid candidate while sharing target information and checking functions. These results describe coverage of this predefined fault model. Type reporting and copying saved valid components do not establish fault localization or repair of unknown generated states.

The exploratory stress test exposes a limit hidden by the primary candidate set: full validation rejects 648 result-equivalent query wrappers, whereas reconstructed decisions without query equality accept 329 of 476 result-changing queries. The practical question is which trusted expectations an interface developer can obtain and how those expectations distinguish permissible alternatives from changed information. Broader contracts require evaluation on held-out and naturally generated updates with independently assessed outcomes. Whether these checks help people inspect and correct an interface remains open.

The 20-update generation pilot identifies two lost time windows and three strict-matching rejections of conforming outputs. Its simpler post-hoc check makes identical decisions. The sequential follow-up reproduces temporal drift in one application-update sequence, but a pre-fixed simple baseline again matches execution-based checking. Generation failures further restrict the attempted cases. We therefore do not recommend interpreting these studies as evidence of added validation capability. Resolving generation-contract inconsistencies and obtaining independent task judgments precede a stronger comparison.

REFERENCES

[1] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (2015), 507–525. https://doi.org/10.1109/TSE.2014.2372785

[2] Yining Cao, Peiling Jiang, and Haijun Xia. 2025. Generative and Malleable User Interfaces with Generative and Evolving Task-Driven Data Model. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 686, 20 pages. https://doi.org/10.1145/3706598.3713285

[3] J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for Bidirectional Tree Transformations: A Linguistic Approach to the View-Update Problem. *ACM Transactions on Programming Languages and Systems* 29, 3, Article 17 (2007). https://doi.org/10.1145/1232420.1232424

[4] Xiao He, Xing Chen, Sibo Cai, Ying Zhang, and Gang Huang. 2018. Testing Bidirectional Model Transformation Using Metamorphic Testing. *Information and Software Technology* 104 (2018), 109–129. https://doi.org/10.1016/j.infsof.2018.07.010

[5] Alistair Johnson, Lucas Bulgarelli, Tom Pollard, Steven Horng, Leo Anthony Celi, and Roger Mark. 2023. MIMIC-IV Clinical Database Demo. *PhysioNet* (2023). https://doi.org/10.13026/dp1f-ex47 Version 2.2.

[6] Zoe Kaputa, Anika Rajaram, Vryan Feliciano, Zhuoyue Lyu, Maneesh Agrawala, and Hariharan Subramonyam. 2026. SimStep: Human-in-the-Loop Authoring of Interactive Educational Simulations Through Task-Level Abstractions. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 75, 34 pages. https://doi.org/10.1145/3772318.3791514

[7] Gyubok Lee, Sunjun Kweon, Seongsu Bae, and Edward Choi. 2024. Overview of the EHRSQL 2024 Shared Task on Reliable Text-to-SQL Modeling on Electronic Health Records. In *Proceedings of the 6th Clinical Natural Language Processing Workshop*. Association for Computational Linguistics, Mexico City, Mexico, 644–654. https://doi.org/10.18653/v1/2024.clinicalnlp-1.62

[8] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. DirectGPT: A Direct Manipulation Interface to Interact with Large Language Models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 975, 16 pages. https://doi.org/10.1145/3613904.3642462

[9] Seokhyeon Park, Soohyun Lee, Eugene Choi, Hyunwoo Kim, Minkyu Kweon, Yumin Song, and Jinwook Seo. 2026. Bridging Gulfs in UI Generation through Semantic Guidance. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 880, 16 pages. https://doi.org/10.1145/3772318.3791966

[10] Venkatesh Sivaraman, Leigh A. Bukowski, Joel Levin, Jeremy M. Kahn, and Adam Perer. 2023. Ignore, Trust, or Negotiate: Understanding Clinician Acceptance of AI-Based Treatment Recommendations in Health Care. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 754, 18 pages. https://doi.org/10.1145/3544548.3581075

[11] Priyan Vaithilingam, Elena L. Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DynaVis: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. Association for Computing Machinery, New York, NY, USA, Article 217, 17 pages. https://doi.org/10.1145/3613904.3642639

[12] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29